

■ Prompt engineering

🌐

40 languages

▼

Article

Talk

Read

Edit

View history

Tools

▼

From Wikipedia, the free encyclopedia

Prompt engineering is the process of structuring [natural language inputs](#) (known as **prompts**) to produce specified outputs from a [generative artificial intelligence](#) (GenAI) model. **Context engineering** is the related area of [software engineering](#) that focuses on the management of non-prompt contexts supplied to the GenAI model, such as [metadata](#), [API](#) tools, and tokens.

It can also be defined as the practice of designing and refining input instructions given to a generative AI model in order to produce more accurate, relevant, or useful outputs. Effective prompt engineering involves understanding how a model interprets language, and may include techniques such as few-shot prompting, chain-of-thought prompting, and role assignment. It is increasingly considered a key skill for working with [large language models](#) (LLMs) in both research and professional contexts.

During the 2020s [AI boom](#), prompt engineering became regarded as an important business capability across corporations and industries. Employees with the title **prompt engineer** were hired to create prompts that would increase productivity and efficacy, although the individual title has since lost traction in light of AI models that produce better prompts than humans and corporate training in prompting for general employees.

Common prompting techniques include multi-shot, chain-of-thought, and tree-of-thought prompting, as well as the use of assigning roles to the model. Automated prompt generation methods, such as [retrieval-augmented generation](#) (RAG), provide for greater accuracy and a wider scope of functions for prompt engineers. [Prompt injection](#) is a type of [cybersecurity](#) attack that targets [machine learning](#) models through malicious prompts.

Terminology [edit]

The Oxford English Dictionary defines prompt engineering as "The action or process of formulating and refining prompts for an [artificial intelligence](#) program, algorithm, etc., in order to optimize its output or to achieve a desired outcome; the discipline or profession concerned with this."^[1] In 2023, prompt ("an instruction given to an artificial intelligence program, algorithm, etc., which determines or influences the content it generates") was the runner-up to Oxford's [word of the year](#).^[2]

Prompt [edit]

A prompt is some [natural language](#) text that describes and prescribes the task that an artificial intelligence (AI) should perform.^[3] A prompt for a text-to-text [language model](#) can be a query, a [command](#), or a longer statement referencing context, [instructions](#), and conversation history. The process of prompt engineering may involve designing clear queries, refining wording, providing relevant context, specifying the style of output, and assigning a character for the AI to mimic in order to guide the model toward more accurate, useful, and consistent responses.^{[4][5]}

When communicating with a [text-to-image](#) or a text-to-audio model, a typical prompt contains a description of a desired output such as "a high-quality photo of an astronaut riding a horse"^[6] or "Lo-fi slow BPM electro chill with organic samples".^[7] Prompt engineering may be applied to text-to-image models to achieve a desired subject, style, layout, lighting, and aesthetic.^[8]

Techniques [edit]

Common terms used to describe various specific prompt engineering techniques include *chain-of-thought*,^[9] *tree-of-thought*,^[10] and [retrieval-augmented generation](#) (RAG).^[11] A 2024 survey of the field identified over 50 distinct text-based prompting techniques, 40 multimodal variants, and a vocabulary of 33 terms used across prompting research, highlighting a present lack of standardised terminology for prompt engineering.^[12]

[Vibe coding](#) is an [AI-assisted software development](#) method where a user prompts an LLM with a description of what they want and lets it generate or edit the code. In 2025, "vibe coding" was the [Collins Dictionary](#) word of the year.^[13]

Context engineering [edit]

Context engineering is a related process that focuses on the context elements that accompany user prompts, which include system instructions, retrieved knowledge, tool definitions, conversation summaries, and task metadata. Context engineering is performed to improve reliability, provenance and token efficiency in [production](#) LLM systems.^{[14][15]} The concept emphasises operational practices such as token budgeting, provenance tags, versioning of context artifacts, observability (logging which context was supplied), and context regression tests to ensure that changes to supplied context do not silently alter system behaviour.^[16]

Rationale [edit]

Research has found that the performance of [large language models](#) (LLMs) is highly sensitive to choices such as the ordering of examples, the quality of demonstration labels, and even small variations in phrasing. In some cases, reordering examples in a prompt produced accuracy shifts of more than 40 percent.^[12]

In-context learning [edit]

A model's ability to temporarily learn from prompts is known as *in-context learning*. In-context learning is an [emergent ability](#)^[17] of large language models. It is an emergent property of model scale, meaning that breaks in [scaling laws](#) occur, leading to its efficacy increasing at a different rate in larger models than in smaller models.^{[17][18]} Unlike training and [fine-tuning](#), which produce lasting changes, in-context learning is temporary.^[19] Training models to perform in-context learning can be viewed as a form of [meta-learning](#), or "learning to learn".^[20]

Prompting to estimate model sensitivity [edit]

Research consistently demonstrates that LLMs are highly sensitive to subtle variations in prompt formatting, structure, and linguistic properties. Some studies have shown up to 76 accuracy points across formatting changes in few-shot settings.^[21] Linguistic features significantly influence prompt effectiveness—such as morphology, syntax, and lexico-semantic changes—which meaningfully enhance task performance across a variety of tasks.^{[5][22]} Clausal syntax, for example, improves consistency and reduces uncertainty in knowledge retrieval.^[23] This sensitivity persists even with larger model sizes, additional few-shot examples, or instruction tuning.

To address sensitivity of models and make them more robust, several evaluative methods have been proposed. FormatSpread facilitates systematic analysis by evaluating a range of plausible prompt formats, offering a more comprehensive performance interval.^[21] Similarly, PromptEval estimates performance distributions across diverse prompts, enabling robust metrics such as performance quantiles and accurate evaluations under constrained budgets.^[24]

Prompting techniques ^[edit]

Multi-shot ^[edit]

A prompt may include a few examples for a model to learn from in context, an approach called *few-shot learning*.^{[25][9]} For example, the prompt may ask the model to complete "*maison* → house, *chat* → cat, *chien* →", with the expected response being *dog*.^[26]

Chain-of-thought ^[edit]

See also: *Reflection (artificial intelligence)*

It has been suggested that *Program of Thought Prompting* be **merged** into this section. (**Discuss**) *Proposed since March 2026.*

Chain-of-thought (CoT) prompting is a technique that allows **large language models** (LLMs) to solve a problem as a series of intermediate steps before giving a final answer. In 2022, **Google Brain** reported that chain-of-thought prompting improves **reasoning** ability by inducing the model to answer a multi-step problem with steps of reasoning that mimic a **train of thought**.^{[9][27]} Chain-of-thought techniques were developed to help LLMs handle multi-step reasoning tasks, such as **arithmetic** or **commonsense reasoning** questions.^{[28][29]}

When applied to **PaLM**, a 540 billion parameter **language model**, according to Google, CoT prompting significantly aided the model, allowing it to perform comparably with task-specific **fine-tuned** models on several tasks, achieving **state-of-the-art** results at the time on the GSM8K **mathematical reasoning benchmark**.^[9] It is possible to fine-tune models on CoT reasoning datasets to enhance this capability further and stimulate better **interpretability**.^{[30][31]}

As originally proposed by Google,^[9] each CoT prompt is accompanied by a set of input/output examples—called *exemplars*—to demonstrate the desired model output, making it a *few-shot* prompting technique. However, according to a later paper from researchers at Google and the **University of Tokyo**, simply appending the words "Let's think step-by-step"^[32] was also effective, which allowed for CoT to be employed as a *zero-shot* technique.

Self-consistency ^[edit]

Self-consistency performs several chain-of-thought rollouts, then selects the most commonly reached conclusion out of all the rollouts.^{[33][34]}

Tree-of-thought ^[edit]

Tree-of-thought prompting generalizes chain-of-thought by generating multiple lines of reasoning in parallel, with the ability to backtrack or explore other paths. It can use **tree search algorithms** like **breadth-first**, **depth-first**, or **beam**.^[10]

Text-to-image prompting ^[edit]

See also: *Artificial intelligence visual art § Prompt engineering and sharing*, and *Artificial intelligence visual art*

In 2022, **text-to-image** models like **DALL-E 2**, **Stable Diffusion**, and **Midjourney** were released to the public. These models take text prompts as input and use them to generate images.^{[35][8]} Early text-to-image models typically do not understand negation, grammar and sentence structure in the same way as **large language models**, and may thus require a different set of prompting techniques. The prompt "a party with no cake" may produce an image including a cake.^[36]

A text-to-image prompt commonly includes a description of the subject of the art, the desired medium (such as *digital painting* or *photography*), style (such as *hyperrealistic* or *pop-art*), lighting (such as *rim lighting* or *crepuscular rays*), color, and texture.^[37] Word order also affects the output of a text-to-image prompt. Words closer to the start of a prompt may be emphasized more heavily.^[38]

Artist styles ^[edit]

Some text-to-image models are capable of imitating the style of particular artists by name. For example, the phrase *in the style of Greg Rutkowski* has been used in Stable Diffusion and Midjourney prompts to generate images in the distinctive style of Polish digital artist **Greg Rutkowski**.^[39] Famous artists such as **Vincent van Gogh** and **Salvador Dalí** have also been used for styling and testing.^[40]

Textual inversion and embeddings ^[edit]

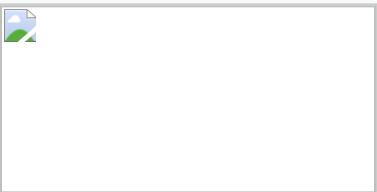
For text-to-image models, *textual inversion* performs an optimization process to create a new **word embedding** based on a set of example images. This embedding vector acts as a "pseudo-word" which can be included in a prompt to express the content or style of the examples.^[41]

Image prompting ^[edit]

In 2023, **Meta**'s AI research released Segment Anything, a **computer vision** model that can perform **image segmentation** by prompting. As an alternative to text prompts, Segment Anything can accept bounding boxes, segmentation masks, and foreground/background points.^[42]

Limitations ^[edit]

The process of writing and refining a prompt for an LLM or generative AI shares some parallels with an iterative engineering design process, such as by discovering reusable best practices through reproducible experimentation. But the techniques that improve performance depend heavily on the specific model being used. Such patterns are also volatile and exhibit significantly different results from seemingly insignificant prompt changes.^{[43][44]}



Demonstration of the effect of negative prompts on images generated with **Stable Diffusion**
Top: no negative prompt
Centre: "green trees"
Bottom: "round stones, round rocks"

Automated prompt generation [[edit](#)]

Recent research has explored automated prompt engineering, using optimization algorithms to generate or refine prompts without human intervention. These automated approaches aim to identify effective prompt patterns by analyzing model gradients, reinforcement feedback, or evolutionary processes, reducing the need for manual experimentation.^[45]

Retrieval-augmented generation (RAG) [[edit](#)]

Retrieval-augmented generation is a technique that enables GenAI models to retrieve and incorporate new information. It modifies interactions with an LLM so that the model responds to user queries with reference to a specified set of documents, using this information to supplement information from its pre-existing **training data**. This allows LLMs to use domain-specific and/or updated information.^[11]

RAG improves large language models by incorporating **information retrieval** before generating responses. Unlike traditional LLMs that rely on static training data, RAG pulls relevant text from databases, uploaded documents, or web sources. By dynamically retrieving information, RAG enables AI to generate more accurate responses and fewer **AI hallucinations** without frequent retraining.^[46]

Graph retrieval-augmented generation (GraphRAG) [[edit](#)]

GraphRAG (coined by **Microsoft Research**) is a technique that extends RAG with the use of a **knowledge graph** to allow the model to connect disparate pieces of information, synthesize insights, and understand summarized semantic concepts over large data collections. It was shown to be effective on datasets like the Violent Incident Information from News Articles.^{[47][48][49]}

Using language models to generate prompts [[edit](#)]

LLMs themselves can be used to compose prompts for LLMs.^[50] The *automatic prompt engineer* algorithm uses one LLM to **beam search** over prompts for another LLM:^{[51][52]}

- There are two LLMs. One is the target LLM, and another is the prompting LLM.
- Prompting LLM is presented with example input-output pairs, and asked to generate instructions that could have caused a model following the instructions to generate the outputs, given the inputs.
- Each of the generated instructions is used to prompt the target LLM, followed by each of the inputs. The log-probabilities of the outputs are computed and added. This is the score of the instruction.
- The highest-scored instructions are given to the prompting LLM for further variations.
- Repeat until some stopping criteria is reached, then output the highest-scored instructions.

CoT examples can be generated by LLM themselves. In "auto-CoT", a library of questions are converted to vectors by a model such as **BERT**. The question vectors are **clustered**. Questions close to the **centroid** of each cluster are selected, in order to have a subset of diverse questions. An LLM does zero-shot CoT on each selected question. The question and the corresponding CoT answer are added to a dataset of demonstrations. These diverse demonstrations can then added to prompts for few-shot learning.^[53]

Automatic prompt optimization [[edit](#)]

Automatic prompt optimization techniques refine prompts for large language models by automatically searching over alternative prompt strings using evaluation datasets and task-specific metrics:

- MIPRO (Multi-prompt Instruction Proposal Optimizer) optimizes the instructions and few-shot demonstrations of multi-stage language model programs, proposing small changes to module prompts and retaining those that improve a downstream performance metric without access to module-level labels or gradients.^[54]
- GEPA (Genetic-Pareto) is a reflective prompt optimizer for compound AI systems that combines language-model-based analysis of execution traces and textual feedback with a Pareto-based evolutionary search over a population of candidate systems; across four tasks, GEPA reports average gains of about 10% over **reinforcement-learning**-based Group Relative Policy Optimization (GRPO) and over 10% over the MIPROv2 prompt optimizer, while using up to 35 times fewer rollouts than GRPO.^[55]
- Open-source frameworks such as DSPy and Opik expose these and related optimizers, allowing prompt search to be expressed as part of a programmatic pipeline rather than through manual trial and error.^{[56][57]}

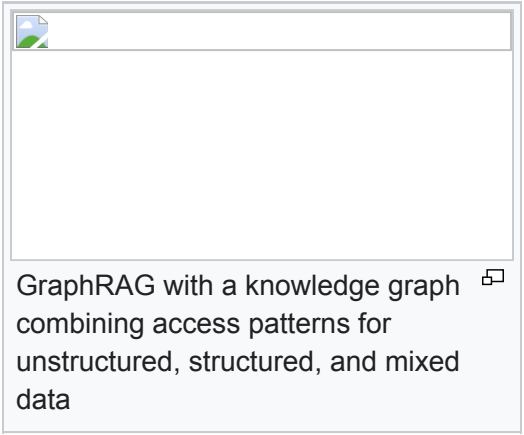
Using gradient descent to search for prompts [[edit](#)]

In "prefix-tuning",^[58] "prompt tuning", or "soft prompting",^[59] **floating-point** vectors are searched directly by **gradient descent** to maximize the log-likelihood on outputs. An earlier result uses the same idea of gradient descent search, but is designed for masked language models like BERT, and searches only over token sequences, rather than numerical vectors. Formally, it searches for **$\arg \max_{\tilde{X}} \sum_i \log Pr[Y^i | \tilde{X} * X^i]$** where $\tilde{X}$ ranges over token sequences of a specified length.^[60]

History [[edit](#)]

Early precedents of structured user interaction with ruled-based AI systems can be found in enterprise automation software from 1990s. For example, The Intelligent Filing Manager (1999), developed by Krishna C. Mukherjee, used a dynamic Q&A interface driven by rule-based **expert system** to collect user inputs for generating regulatory filings automatically across jurisdictions.^[61] While not involving neural networks, such systems featured prompt-like workflows that influenced later human-in-the-loop AI designs. In 2018, researchers first proposed that all previously separate tasks in **natural language processing** (NLP) could be cast as question-answer problems over a context. In addition, they trained a first single, joint, multi-task model that would answer any task-related question like "What is the sentiment" or "Translate this sentence to German" or "Who is the president?"^[62]

The **AI boom** saw an increased focus within academic literature and professional practice on applying prompting techniques to get the model to output the desired outcome and avoid **nonsensical output**, a process characterized by **trial-and-error**.^[63] After the release of **ChatGPT** in 2022, prompt engineering was soon seen as an important business skill; companies began hiring dedicated prompt engineers, although, given advances in AI's ability to generate prompts better than humans, the employment market for prompt engineers has faced uncertainty.^[4] According to ***The Wall Street Journal*** in 2025, the job of prompt engineer was one of the hottest in 2023, but has become obsolete due to models that better intuit user intent and to company trainings.^[64]



A repository for prompts reported that over 2,000 public prompts for around 170 datasets were available in February 2022.^[65] In 2022, the *chain-of-thought* prompting technique was proposed by Google researchers.^{[9][66]} In 2023, several text-to-text and text-to-image prompt databases were made publicly available.^{[67][68]} The Personalized Image-Prompt (PIP) dataset, a generated image-text dataset that has been categorized by 3,115 users, has also been made available publicly in 2024.^[69]

Prompt injection

[edit]

*Main article: **Prompt injection***

*See also: **SQL injection**, **Cross-site scripting**, and **Social engineering (security)***

Prompt injection is a **cybersecurity** exploit in which adversaries craft inputs that appear legitimate but are designed to cause unintended behavior in **machine learning models**, particularly large language models. This attack takes advantage of the model's inability to distinguish between developer-defined prompts and user inputs, allowing adversaries to bypass safeguards and influence model behaviour. While LLMs are designed to follow trusted instructions, they can be manipulated into carrying out unintended responses through carefully crafted inputs.^{[70][71]}

References

[edit]

-
-
-
-

</

40. [^] Solomon, Tessa (August 28, 2024). "The AI-Powered Ask Dalí and Hello Vincent Installations Raise Uncomfortable Questions about Ventriloquizing the Dead" [↗](#). *ARTnews.com*. Retrieved January 10, 2025.
41. [^] Gal, Rinon; Alaluf, Yuval; Atzmon, Yuval; Patashnik, Or; Bermano, Amit H.; Chechik, Gal; Cohen-Or, Daniel (2023). "An Image is Worth One Word: Personalizing Text-to-Image Generation using Textual Inversion". *ICLR*. [arXiv:2208.01618](#) [🔗](#). "Using only 3-5 images of a user-provided concept, like an object or a style, we learn to represent it through new "words" in the embedding space of a frozen text-to-image model."
42. [^] ["Segment Anything"](#) [📄](#) (PDF). ICCV. 2023.
43. [^] Meincke, Lennart; Mollick, Ethan R.; Mollick, Lilach; Shapiro, Dan (March 4, 2025). [Prompting Science Report 1: Prompt Engineering is Complicated and Contingent](#) [↗](#) (Report).
44. [^] ["AI is already eating its own': Prompt engineering is quickly going extinct"](#) [↗](#). *Fast Company*. May 6, 2025.
45. [^] Li, Wenwu; Wang, Xiangfeng; Li, Wenhao; Jin, Bo (2025). "A Survey of Automatic Prompt Engineering: An Optimization Perspective". [arXiv:2502.11560](#) [🔗](#) [cs.AI [↗](#)].
46. [^] ["Can a technology called RAG keep AI models from making stuff up?"](#) [↗](#). *Ars Technica*. June 6, 2024. Retrieved March 7, 2025.
47. [^] Larson, Jonathan; Truitt, Steven (February 13, 2024), [GraphRAG: Unlocking LLM discovery on narrative private data](#) [↗](#), Microsoft
48. [^] ["An Introduction to Graph RAG"](#) [↗](#). *KDnuggets*. Retrieved May 9, 2025.
49. [^] Sequeda, Juan; Allemang, Dean; Jacob, Bryon (2023). "A Benchmark to Understand the Role of Knowledge Graphs on Large Language Model's Accuracy for Question Answering on Enterprise SQL Databases". *Grades-Nda*. [arXiv:2311.07509](#) [🔗](#).
50. [^] [Explaining Patterns in Data with Language Models via Interpretable Autoprompting](#) [📄](#) (PDF). BlackboxNLP Workshop. 2023. [arXiv:2210.01848](#) [🔗](#).
51. [^] *Large Language Models are Human-Level Prompt Engineers*. ICLR. 2023. [arXiv:2211.01910](#) [🔗](#).
52. [^] Pryzant, Reid; Iter, Dan; Li, Jerry; Lee, Yin Tat; Zhu, Chenguang; Zeng, Michael (2023). ["Automatic Prompt Optimization with "Gradient Descent" and Beam Search"](#) [↗](#). *Conference on Empirical Methods in Natural Language Processing*: 7957–7968. [arXiv:2305.03495](#) [🔗](#). doi:10.18653/v1/2023.emnlp-main.494 [↗](#).
53. [^] *Automatic Chain of Thought Prompting in Large Language Models*. ICLR. 2023. [arXiv:2210.03493](#) [🔗](#).
54. [^] Opsahl-Ong, Krista; Ryan, Michael J.; Purtell, Josh; Broman, David; Potts, Christopher; Zaharia, Matei; Khattab, Omar (2024). *Optimizing Instructions and Demonstrations for Multi-Stage Language Model Programs*. Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP). Miami, Florida: Association for Computational Linguistics. [arXiv:2406.11695](#) [🔗](#). doi:10.18653/v1/2024.emnlp-main.525 [↗](#).
55. [^] Agrawal, Lakshya A. (2025). "GEPA: Reflective Prompt Evolution Can Outperform Reinforcement Learning". [arXiv:2507.19457](#) [🔗](#) [cs.CL [↗](#)].
56. [^] Khattab, Omar (2023). "DSPy: Compiling Declarative Language Model Calls into Self-Improving Pipelines". [arXiv:2310.03714](#) [🔗](#) [cs.CL [↗](#)].
57. [^] ["Agent Optimization"](#) [↗](#). *comet.com*. Retrieved November 29, 2025.
58. [^] Li, Xiang Lisa; Liang, Percy (2021). "Prefix-Tuning: Optimizing Continuous Prompts for Generation". *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*. pp. 4582–4597. doi:10.18653/v1/2021.acl-long.353 [↗](#). S2CID 230433941 [↗](#). "In this paper, we propose prefix-tuning, a lightweight alternative to fine-tuning... Prefix-tuning draws inspiration from prompting"

59. [^] Lester, Brian; Al-Rfou, Rami; Constant, Noah (2021). "The Power of Scale for Parameter-Efficient Prompt Tuning". *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*. pp. 3045–3059. [arXiv:2104.08691](#) . [doi:10.18653/V1/2021.EMNLP-MAIN.243](#) . [S2CID 233296808](#) . "In this work, we explore "prompt tuning," a simple yet effective mechanism for learning "soft prompts"...Unlike the discrete text prompts used by GPT-3, soft prompts are learned through back-propagation"
60. [^] Shin, Taylor; Razeghi, Yasaman; Logan IV, Robert L.; Wallace, Eric; Singh, Sameer (November 2020). "AutoPrompt: Eliciting Knowledge from Language Models with Automatically Generated Prompts" . *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Online: Association for Computational Linguistics. pp. 4222–4235. [doi:10.18653/v1/2020.emnlp-main.346](#) . [S2CID 22622232](#) .
61. [^] Mukherjee, Krishna C. (1999). *Automating Forms Publishing with the Intelligent Filing Manager* . IEEE International Conference on Systems, Man, and Cybernetics. pp. 378–383. [doi:10.1109/ICSMC.1999.814108](#) .
62. [^] McCann, Bryan; Keskar, Nitish; Xiong, Caiming; Socher, Richard (June 20, 2018). *The Natural Language Decathlon: Multitask Learning as Question Answering*. ICLR. [arXiv:1806.08730](#) .
63. [^] Knoth, Nils; Tolzin, Antonia; Janson, Andreas; Leimeister, Jan Marco (June 1, 2024). "AI literacy and its implications for prompt engineering strategies" . *Computers and Education: Artificial Intelligence*. **6** 100225. [doi:10.1016/j.caeai.2024.100225](#) . [ISSN 2666-920X](#) .
64. [^] Bousquette, Isabelle (April 25, 2025). "The Hottest AI Job of 2023 Is Already Obsolete" . *Wall Street Journal*. [ISSN 0099-9660](#) . Retrieved May 7, 2025.
65. [^] *PromptSource: An Integrated Development Environment and Repository for Natural Language Prompts* . Association for Computational Linguistics. 2022.
66. [^] Brubaker, Ben (March 21, 2024). "How Chain-of-Thought Reasoning Helps Neural Networks Compute" . *Quanta Magazine*. Retrieved May 9, 2025.
67. [^] Chen, Brian X. (June 23, 2023). "How to Turn Your Chatbot Into a Life Coach" . *The New York Times*.
68. [^] Chen, Brian X. (May 25, 2023). "Get the Best From ChatGPT With These Golden Prompts" . *The New York Times*. [ISSN 0362-4331](#) . Retrieved August 16, 2023.
69. [^] Chen, Zijie; Zhang, Lichao; Weng, Fangsheng; Pan, Lili; Lan, Zhenzhong (June 16, 2024). "Tailored Visions: Enhancing Text-to-Image Generation with Personalized Prompt Rewriting" . *2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE. pp. 7727–7736. [arXiv:2310.08129](#) . [doi:10.1109/cvpr52733.2024.00738](#) . [ISBN 979-8-3503-5300-6](#).
70. [^] Vigliarolo, Brandon (September 19, 2022). "GPT-3 'prompt injection' attack causes bot bad manners" . *The Register*. Retrieved February 9, 2023.
71. [^] "What is a prompt injection attack?" . IBM. March 26, 2024. Retrieved March 7, 2025.



Scholia has a *topic* profile for **Prompt engineering**.

 V•T•E	Generative AI	[show]
 V•T•E	Artificial intelligence (AI)	[show]

Categories: [Deep learning](#) | [Machine learning](#) | [Natural language processing](#) | [Unsupervised learning](#) | [2022 neologisms](#) | [Linguistics](#) | [Generative AI](#)

This page was last edited on 1 April 2026, at 21:22 (UTC).

Text is available under the [Creative Commons Attribution-ShareAlike 4.0 License](#); additional terms may apply. By using this site, you agree to the [Terms of Use](#) and [Privacy Policy](#). Wikipedia® is a registered trademark of the [Wikimedia Foundation, Inc.](#), a non-profit organization.

[Privacy policy](#) [About Wikipedia](#) [Disclaimers](#) [Contact Wikipedia](#) [Legal & safety contacts](#) [Code of Conduct](#) [Developers](#) [Statistics](#) [Cookie statement](#) [Mobile view](#)

--	--